



AVALIAÇÃO P2 · ENGENHARIA DE SOFTWARE

ICEV Remote Sensing Software

Plataforma Web de Sensoriamento Remoto para Agricultura de Precisão

EQUIPE

Luis Gustavo Olimpio · Lauan Matheus · João Leonardi · João Vinícius Castello
Vinícius Henrique · Lucas Benevinuto · José Melquíades

Maio · 2026

Agenda

ROTEIRO DE 20 MIN

- Problema e contexto — agricultura de precisão no Brasil
- Solução — visão geral do produto
- Backlog priorizado e funcionalidades do MVP
- Arquitetura — frontend, backend, dados
- Tecnologias e decisões de design
- Demonstração ao vivo (NDVI, 3D, classificação)
- Estratégia de testes e resultados
- Encerramento, próximos passos e dúvidas

O problema

POR QUE EXISTIR?

Monitorar saúde de culturas é caro, complexo e fragmentado.

- Softwares GIS comerciais (ArcGIS, ENVI) custam milhares de R\$ por ano
- Ferramentas profissionais exigem treinamento especializado
- Ausência de soluções integradas: visualização + análise + classificação
- Imagens Sentinel-2 (gratuitas, 10 m, revisita 5 dias) subutilizadas no Brasil
- Decisões tardias = perdas de produtividade e custos operacionais

A solução

ICEV REMOTE SENSING SOFTWARE

Plataforma web local que processa imagens Sentinel-2 sobre talhões e gera índices, classificações e visualizações 3D — **sem custo de licenciamento**.

Importar Talhões

KML do Google Earth ou desenho direto no mapa

Calcular Índices

NDVI · EVI · SAVI · NDWI · NDBI + 13 bandas

Classificar Zonas

K-Means · Threshold · Supervisionada simulada

Visualizar em 3D

Renderização Three.js com elevação por valor

Para quem é?

USUÁRIOS-ALVO

Produtor Rural

Monitorar lavouras, identificar áreas de estresse, otimizar insumos

Agrônomo

Mapas de variabilidade, prescrição de insumos, análise de vigor

Pesquisador

Experimentar algoritmos sobre dados reais, comparar índices

Estudante

Aprender sensoriamento remoto na prática

Backlog priorizado

6 ÉPICOS · 33 HISTÓRIAS

- Épico 1 — Gestão de Talhões (KML, mapa, desenho) 6 histórias
- Épico 2 — Índices Espectrais (NDVI/EVI/SAVI/NDWI/NDBI/bandas) 9 histórias
- Épico 3 — Visualização & Mapa Interativo (2D, 3D, popup) 5 histórias
- Épico 4 — Lab. de Experimentos (filtros, bordas, morfologia) 6 histórias
- Épico 5 — Classificação (K-Means, Threshold, Supervisionada) 6 histórias
- Épico 6 — Gestão de Dados Sentinel-2 (multi-produto) 1 história

MVP entregue: 28 das 33 histórias (84,8%)

Pendentes: persistência de talhões desenhados · classificação supervisionada real · exportação de experimentos

Arquitetura

VISÃO GERAL

FRONTEND · porta 3000

Next.js 16 · React 19 · TypeScript

- Mapa Leaflet 2D + leaflet-draw
- Three.js / @react-three/fiber (3D)
- shadcn/ui + Tailwind CSS 4

BACKEND · porta 8000

FastAPI + Uvicorn · Python 3.12

- rasterio (JP2 Sentinel-2)
- NumPy · SciPy · scikit-learn · OpenCV
- Shapely · pyproj (geometria, reprojeção)

DADOS LOCAIS

- data/products/ → Sentinel-2 .SAFE (granules em R10m, R20m, R60m)
- data/KML Fields/ → polígonos de talhões (KML do Google Earth)

Stack tecnológica

POR QUE ESCOLHEMOS CADA UMA

FastAPI	Validação Pydantic automática, OpenAPI grátis, async nativo
rasterio	Padrão de fato em I/O geoespacial Python (wrapper sobre GDAL)
Next.js 16 + React 19	App Router unifica páginas e rotas API server-side
Leaflet	Maduro, leve, integra com Esri/OSM sem token pago
Three.js / R3F	Visualização 3D nativa em WebGL, sem peso de Cesium
Tailwind + shadcn/ui	Componentes acessíveis (Radix), zero CSS-in-JS
uv	Build reproduzível e $\sim 10\times$ mais rápido que pip

Decisões de design

TRADE-OFFS EXPLÍCITOS

- **DA-01** · Frontend e backend em processos separados
Permite que o processamento Python pesado não bloqueie a UI
- **DA-02** · Imagens via base64 dentro do JSON
API simples, uma resposta = todos os dados; trade-off: +33% de tamanho
- **DA-04** · Reprojeção dinâmica (WGS84 → UTM)
Frontend trabalha em lat/lon, backend trata projeções via pyproj
- **DA-06** · SentinelProcessor isolado das rotas HTTP
Permite testes unitários sem subir o servidor
- **DA-07** · Tipagem end-to-end (TS + Pydantic)
Erros de contrato detectados em build, não em produção
- **DA-09** · Suavização gaussiana com máscara
Evita “sangramento” de valores nas bordas do polígono

Fluxo de cálculo de NDVI

DIAGRAMA DE SEQUÊNCIA

1. Usuário seleciona talhão e índice “NDVI” na UI
2. Frontend envia `POST /calculate-index` com coordenadas WGS84
3. Backend resolve produto Sentinel-2 .SAFE mais recente
4. rasterio lê bandas B08 (NIR) e B04 (Red) em 10 m
5. pyproj reprojeta polígono WGS84 → CRS nativo da imagem (UTM)
6. rasterio.mask recorta as bandas ao polígono
7. NumPy calcula $NDVI = (NIR - Red) / (NIR + Red)$, clipa a $[-1, 1]$
8. Pillow gera PNG colorido (gradiente **marrom** → **amarelo** → **verde**)
9. Backend retorna JSON com base64, estatísticas, histograma e dados 3D
10. Frontend sobrepõe imagem ao mapa Leaflet + atualiza painel de stats

Demonstração ao vivo

6 MINUTOS

1. Abrir `http://localhost:3000` → selecionar talhão “crop field 1”
2. Calcular NDVI → ver overlay colorido + stats + histograma
3. Clicar no mapa → popup com nível de estresse
4. Trocar para EVI → comparar resultado
5. Ativar visualização 3D → rotacionar terreno
6. Ir em `/classification` → rodar K-Means com 3 classes → hectares por classe

Caso a internet falhe, prints da execução estão em docs/

Estratégia de testes

PLANO EM TRÊS NÍVEIS

UNITÁRIO

pytest + NumPy

- Cálculo NDVI/EVI
- Estatísticas
- Histograma
- Casos de borda

INTEGRAÇÃO

FastAPI TestClient

- /, /health, /products
- Validação Pydantic (422)
- Erros 400 / 404 / 500
- Conversão de coords

MANUAL

Demo end-to-end

- Sentinel-2 real (1 GB)
- Frontend Next.js
- Renderização 3D
- Apresentação ao vivo

Fixtures sintéticas (NumPy) substituem produtos reais — suite roda em < 1 s

```
$ uv run pytest tests/ -v --cov=api
```

Resultados dos testes

EXECUÇÃO DE 09/05/2026

28

casos executados

100%

taxa de sucesso

0,79 s

tempo total

55%

cobertura do core

Cobertura por requisito

- RF-04 (produtos) · RF-05 (NDVI) · RF-06 (EVI) · RF-10 (stats)
- RF-11 (histograma) · RF-20 (health)
- RNF-04 (confiabilidade) · RNF-05 (manutenibilidade)
- Validação Pydantic exercitada com payloads inválidos (HTTP 422)

Limitações conhecidas

O QUE NÃO ESTÁ NO MVP

- **Sem persistência de dados** · talhões desenhados são perdidos ao recarregar
- **Classificação supervisionada simulada** · não usa modelo treinado real
- **Imagens Sentinel-2 baixadas manualmente** · sem download automático via API Copernicus
- **Single-user em localhost** · sem autenticação ou multi-tenancy
- **Cobertura de I/O limitada** · read_band/crop dependem de .SAFE real, testes manuais cobrem

Próximos passos

ROADMAP PÓS-MVP

- Integração com a API do Copernicus (download automático)
- Persistência via SQLite ou PostgreSQL/PostGIS
- Modelo Random Forest treinado para classificação supervisionada real
- Exportação dos resultados em GeoTIFF / Shapefile / CSV
- Comparação temporal (multi-data) lado a lado
- Aplicação web hospedada (multi-usuário, autenticação)
- Pipeline CI com pytest + GitHub Actions



OBRIGADO

Perguntas e demonstração

[github.com / icev-remote-sensing-software](https://github.com/icev-remote-sensing-software)

ENTREGÁVEIS DA P2

MVP funcional · Plano e relatório de testes · Manual do usuário · Documentação técnica
· Apresentação